

CS1010E: Programming Methodology

Assignment 2

Compulsory Questions: Q6 and Q9

Instructions

Submission Instructions

Read **ALL** instructions carefully.

1. Submission Content:

- Each question will be given a template file (e.g., `aN_qM_template.py`).
 - You should work on your file locally before submitting to Coursemology.
- Comment out all the given **test cases** and **import** statements before copying the content of the entire file to Coursemology, *replacing* any content on Coursemology:
 - On template file: **CTRL + A** followed by **CTRL + C** (i.e., *copy all*).
 - On Coursemology: **CTRL + A** followed by **CTRL + V** (i.e., *replace all*).
- If any implementation is given, you are not allowed to change them.
- You are not allowed to *add, remove, or modify* any parameters from functions.
- You are not allowed to change the *name* of the given function.
- Unless otherwise stated, you are not allowed to add/change **import**.
- **You are not allowed to use syntax not covered in the module.**

2. Finalizing Submissions:

- **Remember to click 'FINALIZE SUBMISSION' button before the submission deadline. Otherwise your programs are not submitted and no marks will be awarded.**
- Anyone who fails to submit the programs by the deadline will receive zero mark for the assignment.
- You should not submit at the last minute since Coursemology is not designed to handle large workload.
- If you see that Coursemology is *under maintenance*, it is due to large workload and not an actual maintenance. No extension will be given.

3. Testing:

- Do not call the required functions when you copy the code to Coursemology as it will be called automatically during testing.
- Your function should **return** the result instead of **print**.
- Test cases are called in succession (i.e., *no reverting of global state*).
- Passing all test cases (*public or private*) does not guarantee full marks as we will do manual evaluation on other aspects such as code quality.
- You may assume that the code will only be tested on valid inputs that follows assumption. Do NOT put any checks for inputs with respect to the assumptions.

4. Good Programming Practice:

- Codes that are redundant or too long may be penalized.
- Restrictions must not be violated.

- Each test case (*public or private*) should complete within a reasonable amount of time, typically within 1s to 2s.

Failure to follow any of the instructions above will result in deduction of your marks.

Files

- Assignment02.pdf
- a2_q1_template.py
- ⋮
- a2_q12_template.py

Coursemology

- Assignments → Assignment 2 (2 CA Marks)

Questions

1. Make Out Word
2. Number Concatenation
3. Digital Characters
4. Square Odd Integers
5. Is Tuple Sorted?
6. Max and Min [1 CA Mark]
7. Standard Deviation
8. Anagram
9. Merged Sorted Tuples [1 CA Mark]
10. Resistors
11. Ancestry Tree
12. Count Substring

Question 1: Make Out Word

Given a string `s` of exactly 4 characters (e.g. `'<<>>'`) and another string `word` (e.g. `'hi'`), the function `make_out_word(s, word)` returns a **new string** in which `word` is in the middle of `s`, e.g. `'<<hi>>'`.

More examples:

- Function call `make_out_word('<<>>', 'Yay')` returns `'<<Yay>>'`
- Function call `make_out_word('<<>>', 'WooHoo')` returns `'<<WooHoo>>'`
- Function call `make_out_word('[]', 'word')` returns `'[[word]]'`

Hint: Use string slicing wisely and there is no need to use any loop.

Question 2: Number Concatenation

Write a function `concat(m, n)` that takes two integers `m` and `n` as parameters, returns an integer in which `m` and `n` are 'concatenated'. You may assume that `m` is not zero and the return value of the function call is not overly large.

For example, function call `concat(1234, 567)` returns integer `1234567`. Note that the return value of the function call is an integer (i.e. `int` type).

Hint: The easiest way is to use string concatenation.

Question 3: Digital Characters

Write a function `at_most_10(string)` that checks if the given `string` contains at most 10 **digit characters**. The function returns `True` if so, or `False` otherwise.

Digit characters refer to characters `'0'`, `'1'`, till `'9'`. Note that string `'10'` contains two digit characters.

Sample Runs

```
1 >>> at_most_10('0123456789')
2 True
3 >>> at_most_10('0!1a2A3b4B5c6C7d8D9EFGHIJ')
4 True
5 >>> at_most_10('a0b1c2d3e4f5g6h7i8j9k10')
6 False
```

Hint: To check if a character `ch` is a digit character, you can use the expression: `ch >= '0' and ch <= '9'` or equivalently `'0' <= ch <= '9'`.

Question 4: Square Odd Integers

Write a function `square_odd_terms(tpl)` that accepts a tuple of integers as parameter and returns a list in which all the odd integers of `tpl` are squared, even integers remain the same, and the order of all integers remain the same.

For example, function call `square_odd_terms( (2, 3, 4, 5) )` returns the list `[2, 9, 4, 25]`.

Question 5: Is Tuple Sorted?

Write a function `is_sorted(tup)` that takes a tuple of integers `tup` as parameter and returns `True` if all the integers are sorted in **non-decreasing** order, or `False` otherwise. You may assume that `tup` contains at least 2 integers.

For example, function call `is_sorted( (1, -2, 8, 6) )` returns `False` and function call `is_sorted( (4, 6, 7, 7, 9) )` returns `True`.

Question 6: Max and Min

[1 CA Mark]

A function can return at most one value. However, sometimes we may want to return multiple values from a function. Tuples can come in handy in such cases. For example, we can create a tuple containing multiple values and return the tuple when function finishes execution.

Write a function `max_and_min(values)` that accepts a tuple of integers as parameter. The function returns a new tuple containing the largest and smallest value in `values`, in that order. You may assume that `values` contains at least one integer.

For example, function call `max_and_min( (1, -1, 7, 0) )` returns tuple `(7, -1)`.

You are **NOT** allowed to use built-in functions `max` and `min` in this question.

Hint: Define two variables `maximum` and `minimum`. Use a loop to go through the tuple element by element; compare and update `maximum` and `minimum` along the way. Also be careful when choosing the initial values of `maximum` and `minimum`.

Question 7: Standard Deviation

Write a function `deviation(tpl)` that accepts a tuple of integers as parameter and returns the standard deviation of all the integers.

The standard deviation `s` is computed according to the following formula:

$$s = \sqrt{\frac{(x_1 - \bar{x})^2 + (x_2 - \bar{x})^2 + \dots + (x_n - \bar{x})^2}{n}}$$

The variable $\bar{x}$ is the average of all the integers in `tpl`.

You are allowed to use (1) Python built-in function `sum()`, and (2) `sqrt()` function from the `math` module.

Question 8: Anagram

An anagram is a word formed by rearranging letters of a different word and using all the original letters exactly once. For instance, `Listen` and `Silent` are anagram because of the mapping shown in Figure 1.



Figure 1: Mapping between `listen` and `silent`.

Write a function `is_anagram(s1, s2)` that returns `True` if two strings `s1` and `s2` are anagram of each other, or `False` otherwise. You may assume that `s1` and `s2` consists of only lowercase or uppercase letters from the English alphabets.

For example,

- function call `is_anagram('ABC', 'BCA')` returns `True`
- function call `is_anagram('ABC', 'bca')` returns `False`

You are **NOT allowed** to use any sorting functions provided by Python. Instead, you are supposed to use **dictionary** to solve this question.

Hint: Use dictionary to note down the occurrence of each letter.

Question 9: Merged Sorted Tuples

[1 CA Mark]

Merge Sort is an advanced sorting technique that will be introduced in second programming module. Here, we are not going to explain how merge sort works, but focus on one idea engaged in merge sort: merging two sorted tuples into a bigger sorted tuple.

For instance, given two sorted tuples `(-1, 1, 3, 5)` and `(-2, 4, 6, 7)`, the merged tuple would be `(-2, -1, 1, 3, 4, 5, 6, 7)`.

Write a function `merge(tup1, tup2)` that accepts two sorted tuples as parameters, and returns the merged tuple in which all integers appears in ascending order.

You may assume that

1. `tup1` and `tup2` each contains distinct integers sorted in ascending order.

2. integers in `tup1` are different from those in `tup2`.

You are **NOT allowed** to do sorting or use any sorting functions provided by Python.

Hint: A smart algorithm just needs a single loop to solve this problem. However, using single loop or nested loop doesn't affect your attempt mark.

Question 10: Resistors

In electrical circuits, a resistor is used to reduce the flow of current. Its capability to reduce the flow of current depends on its resistance value, measured in ohm (Ω).



Figure 2: IEC resistor symbol (www.wikipedia.org).

When connected in series, the total resistance is the sum of each individual resistance values.

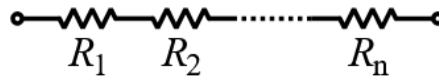


Figure 3: Resistor connected in series, the equivalent resistance value is $R_{eq} = R_1 + R_2 + \dots + R_n$.

As an engineer, you are given a sequence of resistors which are already sorted in ascending order of resistance values.

Write a function `resistors(seq, R)` that takes in a tuple of resistors `seq` and a value `R` as parameter. The function checks if any two resistors in `seq` that add up to `R`. It returns `True` if so, or `False` otherwise.

For example, function call `resistors((1, 3, 4), 4)` returns `True` because $1 + 3 = 4$ and function call `resistors((1, 3, 5, 7), 5)` returns `False`.

You may assume that `seq` contains distinct integers sorted in ascending order.

Note: This question can be done using a nested loop. However, such a design is not the best as the program is complex and execution takes longer time. For strong students, you may challenge yourself to use one single loop to solve this question.

Restrictions

- You are not allowed to modify the input sequence.

Assumptions

- `len(seq) >= 2`
- `seq[i]` contains only positive integers sorted in ascending order

Question 11: Ancestry Tree

All of us may come from different families, but we may share common ancestors! In this question, we will explore the use of a dictionary to model ancestry trees.

Given some ancestry information such as shown in Figure 4, we may list the following information:

- Ben is Amy's parent
- Ben is Tom's parent
- Amy is Frank's parent
- Tom is May's parent
- Howard is Ben's parent
- George is Howard's parent

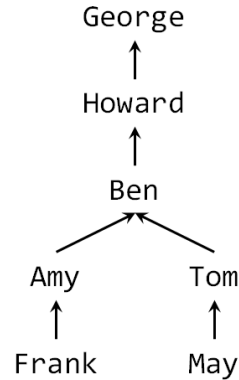


Figure 4: A simple ancestry tree.

We may easily construct the *graphical representation* of the ancestry tree (*right*) and we can also construct a dictionary to represent our ancestry tree:

```

1 tree = {
2     'Amy': 'Ben', 'Tom': 'Ben', 'Frank': 'Amy',
3     'May': 'Tom', 'Ben': 'Howard', 'Howard': 'George'
4 }
```

In the dictionary above, the *keys* of the dictionary represent the *child* while the *values* of the dictionary represent the *parent*. You can check that indeed Ben is Amy's parent since given the key 'Amy' it gives us the value 'Ben' (*i.e.*, `tree['Amy'] == 'Ben'`).

Assumptions

- Every name in the dictionary is unique.
- Every person has at most one parent.
- Your code should work any ancestry tree.
- There will be no cycle (*i.e.*, *I am my own grandparent scenario*).

Restrictions

- The dictionary `tree` should not be modified.

Find your Ancestor

Write a function `find_ancestor(name, tree)` that returns the ancestor of `name`. You may assume that `name` belongs to the ancestry `tree`.

For example, function call `find_ancestor('Amy', tree)` will return `'George'`

Are They Related?

Two people are related if they share a common ancestor in the ancestry tree. In the example ancestry tree given above, any two *distinct* people are related to one another. For instance, Amy and Tom are related since they share a common ancestor George. Additionally, Ben and Howard are related since they share a common ancestor George.

On the other hand, if we introduce a new person Luna with no ties to anyone in the given tree above, then Luna is not related to anyone else. By convention, a person must be related to himself/herself.

Write the function `is_related(name1, name2, tree)` to check if the person `name1` is related to the person `name2` in the ancestry tree `tree`. The function returns `True` if so, or `False` otherwise.

You may assume that `name1` is different from `name2`.

Consider the following modified ancestry tree:

```
1 tree2 = {
2     'Amy' : 'Ben'    , 'Tom'  : 'Ben'    , 'Frank' : 'Amy',
3     'May' : 'Tom'    , 'Ben'  : 'Howard', 'Howard': 'George',
4     'Joe' : 'Bill'   , 'Bill' : 'Mary'   , 'Zoe'   : 'Mary',
5     'Mary': 'Philip', 'Simon': 'Bill'
6 }
```

Sample Runs

```
1 >>> print(find_ancestor('Frank', tree2))
2 George
3 >>> print(is_related('Joe', 'Philip', tree2))
4 True
```

Question 12: Count Substring [Hard]

Write a function `count_substring(string)` that accepts a `string` as parameter, and returns the number of substrings that begin with character `'A'` and ends with character `'X'`.

For example, suppose `string = 'CAXAAYXZA'`, there are four substrings that begin with `'A'` and ends with `'X'`, namely: `'AX'`, `'AXAAYX'`, `'AAYX'`, and `'AYX'`. Therefore, function call `count_substring('CAXAAYXZA')` returns `4`.

You can assume that `string` is composed of upper case letters only.

Note: This question can be done using a nested loop. However, such a design is not the best as the program is complex and execution takes longer time. For strong students, you **may challenge** yourself to use one single loop to solve this question.